Hamdard University Karachi
Faculty of Engineering Sciences and Technology

Department of Information Technology (DIT)

Scripting Programs / Scripting Program and Cloud Architecture

DFSP-5208 (BS Digital Forensics and Cyber Security) | CCSP-5213 (BS Cloud Application Development and Operations)

Complete Lecture Notes, Weeks 1-16

Revised edition for Fall 2026

Semester 6 | 2 + 1 credit hours | Instructor: Iqbal Uddin Khan

Includes: revision log, Weeks 1-16, assessment pack, personalized evidence-based questions

Contents

1. Fall 2026 Revision Summary 6
2. Course Overview and Assessment Map 9
Course learning outcomes 9
Marks distribution (identical in both courses) 9
Sixteen-week teaching map 9
3. Lab Environment and Command Equivalents 11
Part A: Weeks 1-8 (Foundations) 12
Week 1: Introduction to Scripting vs Programming 13
Learning objectives 13
Hour 1: Conceptual foundations 13
Hour 2: Applications in security and cloud 13
Hour 3: Practical orientation 13
Step-by-step lab method (free and open-source tools only) 14
In-class / take-home questions (evidence-based) 14
Week 2: Python Basics: Syntax, Data Types, Loops, Functions 15
Learning objectives 15
Hour 1: Core syntax 15
Hour 2: Functions, loops and dictionaries 15
Hour 3: Hands-on lab 16
In-class / take-home questions (evidence-based) 16
Week 3: Python Scripting for Automation and CLI Tools 17
Learning objectives 17
Hour 1: File handling 17
Hour 2: Command-line tools 17
Hour 3: Practical and Git 17
In-class / take-home questions (evidence-based) 18
Week 4: Bash Scripting Fundamentals 19
Learning objectives 19
Hour 1: Bash basics 19
Hour 2: Pipes and system-level automation 19
Hour 3: Lab 19
In-class / take-home questions (evidence-based) 19
Week 5: Bash for Backups, Evidence Handling and Cloud CLI 21
Learning objectives 21
Hour 1: Advanced Bash 21
Hour 2: Integration with cloud CLI 21
Hour 3: Lab 21
In-class / take-home questions (evidence-based) 21

Week 6: Web Scraping and Threat/Status Data Feeds 23
Learning objectives 23
Hour 1: Web scraping concepts 23
Hour 2: Data storage 23
Hour 3: Lab 23
In-class / take-home questions (evidence-based) 24
Week 7: Node.js Basics for Scripting 25
Learning objectives 25
Hour 1: JavaScript and Node fundamentals 25
Hour 2: Asynchronous programming 25
Hour 3: Lab 25
In-class / take-home questions (evidence-based) 26
Week 8: Node.js Scripting for Automation and Serverless Deployment 27
Learning objectives 27
Hour 1: Serverless concepts 27
Hour 2: Packaging and deployment 27
Hour 3: Lab (choose a tier) 27
In-class / take-home questions (evidence-based) 28
Part B: Weeks 9-16 (APIs, Reliability, Containers, CI/CD) 29
Week 9: Midterm Examination 30
Suggested paper blueprint (20 marks) 30
Week 10: API Interaction: REST APIs and JSON Parsing 31
Learning objectives 31
Hour 1: REST, HTTP and status codes 31
Hour 2: JSON as the data-interchange format 31
Hour 3: Python workflow with requests 32
Step-by-step lab method (free and open-source tools only) 34
In-class / take-home questions (evidence-based) 34
Week 11: Automating Workflows with APIs 36
Learning objectives 36
Hour 1: From single script to workflow 36
Hour 2: Building the multi-step automation 36
Hour 3: State persistence and unattended scheduling 37
Step-by-step lab method (free and open-source tools only) 37
In-class / take-home questions (evidence-based) 38
Week 12: Error Handling, Debugging and Collaboration 39
Learning objectives 39
Hour 1: A taxonomy of errors 39
Hour 2: Debugging beyond print() 40

Hour 3: Node.js error handling and team workflow 40
Step-by-step lab method (free and open-source tools only) 40
In-class / take-home questions (evidence-based) 41
Week 13: Logging and Monitoring 42
Learning objectives 42
Hour 1: Why logging matters 42
Hour 2: Structured logging in Python 42
Hour 3: Winston and an open-source monitoring stack 43
Step-by-step lab method (free and open-source tools only) 44
In-class / take-home questions (evidence-based) 44
Week 14: Introduction to Docker and Containerisation 45
Learning objectives 45
Hour 1: Why containerize 45
Hour 2: Writing the Dockerfile 45
Hour 3: Build, run, share 46
Step-by-step lab method (free and open-source tools only) 46
In-class / take-home questions (evidence-based) 46
Week 15: Scripting for CI/CD Pipelines 48
Learning objectives 48
Hour 1: CI/CD concepts 48
Hour 2: GitHub Actions 48
Hour 3: Test, build, scan 48
Step-by-step lab method (free and open-source tools only) 49
In-class / take-home questions (evidence-based) 50
Week 16: Best Practices and Final Project Presentations 51
Hour 1: Best practices checklist 51
Hours 2-3: Final project presentations 51
Final exam revision plan (Weeks 1-16) 52
Part C: Assessment Pack (Fall 2026) 53
Quiz 1 (Weeks 1-5), 3 marks, closed book, 15 minutes 53
Quiz 2 (Weeks 6-10), 3 marks, closed book, 15 minutes 53
Quiz 3 (Weeks 11-15), 4 marks, closed book, 20 minutes 54
Assignments 55
Assignment 1 (Weeks 1-5), 3 marks, CLO 1 55
Assignment 2 (Weeks 6-10), 3 marks, CLO 2 (2) and CLO 3 (1) 55
Assignment 3 (Weeks 11-16), 4 marks, CLO 2 (2) and CLO 3 (2) 55
One-Hour Class Task (Mid-Semester Practical), 10 marks 57
Final Exam Blueprint (Weeks 1-16), 50 marks 58
Appendix A: One Repository Across Weeks 10-16 59

Appendix B: Personalisation and AI-Use Policy for Evidence-Based Questions....60
Appendix C: Pre-Term Checklist for the Instructor....61
Appendix D: Reference List....62

1. Fall 2026 Revision Summary

This document merges the two course tracks into one teaching pack: **DFSP-5208 Scripting Programs** (BS Digital Forensics and Cyber Security) and **CCSP-5213 Scripting Program and Cloud Architecture** (BS Cloud Application Development and Operations). Both are 2+1 credit hour, Semester 6 courses with the same instructor, the same 16-week structure and the same three CLOs. Where the two tracks differ, the text carries a blue **Forensic** box or a green **Cloud** box. Everything else is shared, so you can teach both sections from one set of slides.

I re-analyses all five files (Weeks 1-8 combined notes, Weeks 9-15 notes for each course, and both approved course outlines) against each other and against the tooling that is current as of October 2026. The table below lists what I found and what I changed. The most urgent item is the first one.

#	Finding	Where it appeared	Change made in this document
1	**LocalStack Community Edition no longer exists as a free anonymous image.** Since 23 March 2026, localstack/localstack requires an account and auth token; the free tier is non-commercial only and the Community image is frozen.	CCSP Weeks 10, 11, 15	Primary emulator is now **Moto** (pip install "moto[server]", no Docker, no account). LocalStack (free Hobby token) and Floci (MIT, no token) are listed as options. Please do a dry run of your chosen emulator before Week 10.
2	Week 8 serverless lab assumes a real AWS Lambda / Azure Functions account (card and billing risk for students).	Both outlines, Week 8	Four-tier lab: local Node harness, emulator Lambda, Azure Functions Core Tools, and optional real cloud for students who already have an account.
3	Suggested reading mismatches: Week 10 cites *Practical Malware Analysis* Ch. 6 (x86 control flow, not REST). Week 13 cites *Docker Deep Dive* Ch. 2-3 for logging. Week 11 cites *Automate the Boring Stuff* Ch. 14, which is Google Sheets in the 2nd edition.	Outlines and notes	Replacement readings suggested (Python docs, Real Python style references, ATBS 2nd ed. Ch. 16-18). Please verify chapter numbers against your edition.
4	The one-hour mid-semester test requires try/except and logging, but those are first taught in Weeks 12-13, after the test.	Both notes, class task	Short **primers** added: exceptions in Week 2, logging basics in Week 3, Bash exit codes in Week 5. Full treatment stays in Weeks 12-13.
5	CLO 3 requires **version control**, but no week teaches Git.	Both outlines	A Git thread now runs through Weeks 1, 3, 7, 12 and 15 (init, commit, branch, pull request, review).
6	Assignment 2 is a **Node.js** API task, but Week 10 teaches only Python requests.	Notes Weeks 10, Assignment 2	Node.js fetch/axios parity added to Week 10.
7	datetime.utcnow() is deprecated from Python 3.12 (the version used in the Docker lab).	Weeks 10, Week 13	Replaced with timezone-aware datetime.now(timezone.utc).

#	Finding	Where it appeared	Change made in this document
8	Week 14 .dockerignore excludes .env, but the workflow script needs the API key at run time; image also runs as root.	Week 14	Added --env-file, non-root USER, pinned base tag, and a "secrets never baked into images" check.
9	Commands are Linux/macOS only (source venv/bin/activate, $(pwd), sha256sum, cron). Most lab PCs run Windows.	Weeks 10-15	Command equivalents table in Section 3 and inline Windows notes.
10	CI workflow uses trivy-action@master and unpinned actions, and the Trivy step never fails the build. This is a supply-chain risk in a security course.	Week 15	Pin actions to tagged releases or commit SHAs, set least-privilege permissions, make Trivy fail on HIGH/CRITICAL.
11	Email alerting depends on a personal mail provider; smtpd was removed in Python 3.12.	Week 11	Webhook alert is primary; aiosmtpd local debug SMTP server added for offline email demos.
12	Week 13 log-hash exercise: a live log changes every time it is written, so start/end hashes always differ. The basicConfig + RotatingFileHandler snippets also conflict.	Week 13	Corrected configuration and a better tamper-evidence exercise (hash a closed/rotated file, store the hash separately).
13	**Week 16 has no lecture notes** although both outlines schedule it.	Both	Week 16 added (best practices, final project presentations, rubric).
14	Quiz 3 says "Weeks 11-16" but draws on "Weeks 10-15"; Assignment 3 rubric omits Git; the class-task CLO mapping differs between sections of the DFSP outline (marks 3/4/3, indicative 30/40/30, but Assessment Methods lists only CLO 2 and 3). The CCSP indicative table says 20/50/30.	Outlines	Assessment pack aligned to the **marks distribution tables** (the approved totals: CLO 1 = 25, CLO 2 = 41, CLO 3 = 34). Please correct the indicative percentage rows at the next outline revision.
15	Weeks 1-8 notes are bullet-level and the scenario questions can be answered by a chatbot in one paste.	Weeks 1-8 notes	Each week expanded with teaching points, a lab, and **evidence-based, personalised** questions. See Appendix C.
16	Free-tier APIs change. A key-based lab can fail on the day.	Week 10	Added a 25-line **instructor mock API** (with a deliberate 429 after three calls) as a guaranteed fallback.

#	Finding	Where it appeared	Change made in this document
17	**Promtail is deprecated** in favour of Grafana Alloy; the Cloud Week 13 step points Promtail at a log file.	CCSP Week 13	Use Grafana Alloy, or push lines to the Loki HTTP API as a simple fallback.

Items I could not verify and you should check before term

1. Current free-tier limits and the exact endpoint/header format of AbuseIPDB (or any alternative API you choose). 2. Chapter numbers in your edition of each textbook. 3. Whether your lab PCs permit Docker (virtualization / WSL 2) and outbound access to Docker Hub, PyPI, npm and GitHub. 4. Current major versions of actions/checkout, actions/setup-python and aquasecurity/trivy-action. 5. Moto's /moto-api/ status endpoint behaviour on the version you install. Fall 2026 calendar dates were not in the files, so the pack uses week numbers only.

2. Course Overview and Assessment Map

Course learning outcomes

CLO	BT level	DFSP-5208 (Forensics)	CCSP-5213 (Cloud)
CLO 1	Apply (L3)	Apply Python, Bash, JavaScript to automate forensic and cybersecurity tasks.	Apply Python, Bash, JavaScript to automate tasks and interact with cloud services.
CLO 2	Create (L6)	Design and deploy security-focused applications using APIs, automation and containerization.	Design and deploy cloud-native applications using containerization, serverless computing and APIs.
CLO 3	Analyze (L4)	Demonstrate debugging, error handling and collaborative project management using version control in forensic scripting projects.	Demonstrate debugging, error handling and collaborative project management using version control in cloud-based scripting projects.

Marks distribution (identical in both courses)

Assessment	Weeks	Marks	CLO 1	CLO 2	CLO 3
Assignment 1	1-5	3	3	0	0
Assignment 2	6-10	3	0	2	1
Assignment 3	11-16	4	0	2	2
Quiz 1	1-5	3	2	1	0
Quiz 2	6-10	3	0	2	1
Quiz 3	11-16	4	0	2	2
One-hour class task	Mid-semester	10	3	4	3
Midterm exam	1-8	20	12	8	0
Final exam	1-16	50	5	20	25
Total		**100**	**25**	**41**	**34**

Sixteen-week teaching map

Wk	Topic	Lab	CLO	Assessment event
1	Scripting vs programming; role in security and cloud	Hello script, environment, Git init	1	Quiz 1 window opens
2	Python basics, functions, files, exception primer	Parse log / CSV	1	
3	Python automation, JSON/CSV, argparse CLI, logging primer	CLI log or CSV tool	1	Assignment 1 issued
4	Bash fundamentals, pipes, cron intro	Directory + cleanup script	1	

Wk	Topic	Lab	CLO	Assessment event
5	Bash for backups and CLI integration, exit codes	Backup with cron + hashing	1	Quiz 1; Assignment 1 due
6	Web scraping and data feeds	Scrape to CSV	1	Assignment 2 issued
7	Node.js basics	File-renaming script	1	
8	Node.js serverless deployment	Local-first serverless lab	2	One-hour class task (suggested)
9	Midterm examination	None	1, 2	Midterm (Weeks 1-8)
10	REST APIs and JSON	Fetch, save, hash	1	Quiz 2; Assignment 2 due
11	Automating workflows with APIs	Fetch-evaluate-alert, SQLite, scheduler	2	Assignment 3 issued
12	Error handling, debugging, Git collaboration	pdb, custom exceptions, PRs	3	
13	Logging and monitoring	Logging, Winston, optional Loki	3	
14	Docker and containerization	Build and run image	2	
15	CI/CD with GitHub Actions	Test, build, scan pipeline	2	Quiz 3
16	Best practices; final presentations	Capstone demos	2, 3	Assignment 3 due; final exam revision

3. Lab Environment and Command Equivalents

Install and test the following **before Week 1** on a sample lab PC and on your own laptop. All tools are free or have a free personal/education tier.

- **Python 3.12 or newer**, VS Code, Git. Packages: requests, python-dotenv, beautifulsoup4, boto3 (Cloud track), moto[server] (Cloud track), aiosmtpd (optional).
- **Node.js current LTS** (22 or 24) with npm. Node 18+ has built-in fetch; axios and winston are installed with npm.
- **Bash**: native on Linux/macOS; on Windows use WSL 2 (Ubuntu) or Git Bash. Weeks 4-5 assume a Linux shell.
- **Docker Engine** (Linux) or **Docker Desktop** (free for education and personal use; check the current terms for the university lab). WSL 2 backend on Windows.
- A free **GitHub** account per student (needed from Week 3). Use a shared GitHub Classroom or organization if the department has one.

Task	Linux / macOS / WSL	Windows (PowerShell)
Create venv	python3 -m venv venv	python -m venv venv
Activate venv	source venv/bin/activate	venv\Scripts\Activate.ps1
SHA-256 of a file	sha256sum file	Get-FileHash file -Algorithm SHA256
Current dir in docker -v	-v "$(pwd)/evidence:/app/evidence"	-v "${PWD}/evidence:/app/evidence"
Schedule every 30 min	crontab -e then */30 * * * * ...	Task Scheduler, or schtasks /create /sc minute /mo 30 /tn Job /tr "python C:\path\workflow.py"
Environment variable (session)	export API_KEY=...	$env:API_KEY = "..."
Show system clock in screenshot	date -u; date	Get-Date; (Get-Date).ToUniversalTime()
Find text in files	grep -rn "ERROR" .	Select-String -Path *.log -Pattern ERROR

Part A: Weeks 1-8 (Foundations)

Weeks 1-8 build the CLO 1 skills (apply scripting) and end with the first CLO 2 experience (serverless deployment). The original notes listed topics; below each week has teaching points, a lab, and evidence-based questions.

Week 1: Introduction to Scripting vs Programming

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Sweigart, Automate the Boring Stuff, Ch. 1	Hello Security / Hello Cloud script

Learning objectives

- Distinguish scripting from compiled programming and say when each is the right tool.
- Explain why automation reduces human error and improves repeatability in security and cloud operations.
- Set up a working environment (Python, VS Code, Git) and run a first script.

Hour 1: Conceptual foundations

- **Scripting vs compiled programming:** scripts are interpreted line by line and optimized for glue work and quick change; compiled languages (C++, Java) are built into binaries for performance and large systems. The line is blurry (Python compiles to bytecode), so teach it as a *purpose* distinction.
- **Why automate:** consistency, speed, auditability, and removal of copy-paste error. A manual step done 500 times will eventually be done wrongly once.
- **Case study:** an analyst parsing thousands of log lines after a breach versus an engineer scripting nightly backups.

Hour 2: Applications in security and cloud

- Digital forensics: log parsing, hashing evidence, malware triage, timeline building.
- Cloud: provisioning, snapshots, API integration, cost and capacity checks.
- DevOps and incident response: the same script runs the same way every time, which is the foundation of Weeks 14-15.

> **Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)**
> Show a 6-line script that hashes every file in a folder. Explain that identical hashes before and after a copy are the evidence-integrity check used throughout the course.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> Show the same loop listing the objects in a storage bucket. Explain that the loop is identical, only the data source changes.

Hour 3: Practical orientation

- Demo: print("Hello Security") / print("Hello Cloud"), running from terminal and from VS Code.
- Create a project folder, a virtual environment and a Git repository (git init, first commit). Explain that every later lab is committed.
- Discussion: where would a script beat a GUI in your own daily work?

```
import platform, datetime, hashlib, pathlib
me = pathlib.Path(__file__).read_bytes()
print("Host :", platform.node())
print("UTC  :", datetime.datetime.now(datetime.timezone.utc).isoformat())
print("SHA256 of this script:", hashlib.sha256(me).hexdigest())
```

Step-by-step lab method (free and open-source tools only)

1. Install Python, VS Code and Git; confirm python --version and git --version.
2. Create week01/, a venv, and git init.
3. Save the script above as hello.py and run it twice, editing one character in between.
4. Observe that the hash changes; commit both versions with meaningful messages.
5. Take a screenshot showing the terminal and the system clock.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalization.

1. Run hello.py and paste the exact output. State your machine name and UTC time as printed.
2. Edit one character of the script and paste both SHA-256 values. Explain in one sentence what this shows about integrity checking.
3. Show git log --oneline from your repository with at least two commits.
4. Describe one repetitive task from your own life or lab that you would script first, and write the first three steps as pseudo-code.
5. In two sentences, argue for a script or a compiled program for a tool that must scan 10 GB of logs in under a minute. Defend the trade-off.

Week 2: Python Basics: Syntax, Data Types, Loops, Functions

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Severance, Python for Everybody, Ch. 2-3	Parse a log (Forensic) or CSV (Cloud)

Revised for Fall 2026

New: a 15-minute **exception primer** (try/except ValueError) so that students can handle malformed lines in Assignment 1 and the class task before Week 12.

Learning objectives

- Use variables, core data types, control flow and functions.
- Read a text file line by line and handle a malformed line without crashing.
- Use dictionaries to count occurrences.

Hour 1: Core syntax

- Dynamic typing; int, float, str, bool, list, dict, set, tuple.
- Control flow: if/elif/else, for, while, break, continue.
- Indentation is syntax. Show one indentation bug and read the error message together.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Case study: lines like 2026-03-04 10:22:01 FAILED LOGIN user=admin ip=203.0.113.9. Which fields do you need?

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Case study: a CSV of cloud resources (id,region,status). Which rows are unhealthy?

Hour 2: Functions, loops and dictionaries

- Write small reusable functions: parse_line(line) returns a dict or None.
- Counting with a dictionary: counts[ip] = counts.get(ip, 0) + 1.
- **Primer:** wrap the parse in try/except ValueError and continue on bad lines; log how many lines were skipped.

```
def parse_line(line):
    parts = line.strip().split()
    if len(parts) < 4:
        return None
    return {"date": parts[0], "time": parts[1], "event": parts[2]}

counts, skipped = {}, 0
with open("sample.log", encoding="utf-8") as f:
    for line in f:
        rec = parse_line(line)
        if rec is None:
            skipped += 1
            continue
        counts[rec["event"]] = counts.get(rec["event"], 0) + 1
print(counts, "skipped:", skipped)
```

Hour 3: Hands-on lab

- Instructor supplies sample.log (Forensic) or resources.csv (Cloud) with a few deliberately broken lines.
- Students count events per type and report the number of skipped lines.

In-class / take-home questions (evidence-based)

> **Evidence rule**
>
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalization.

1. Run your parser on the supplied file and paste the exact counts and the skipped-line number.
2. Show the exact malformed line that your code skipped (print it with its line number).
3. Change the parser to also record the earliest and latest timestamp. Paste the code you added and the output.
4. Explain what would happen to your script if the file did not exist, and show the one-line change that would make it fail gracefully.
5. Your instructor changes one field in the file. Re-run and report which count changed. Seed: use the last two digits of your roll number as the number of lines to filter on, and state it.

Week 3: Python Scripting for Automation and CLI Tools

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Sweigart, Automate the Boring Stuff, Ch. 9 (files)	argparse log or CSV tool

Revised for Fall 2026

New: **logging primer** (10 minutes, logging.basicConfig, replacing print) and **Git basics** (commit, branch, push to GitHub) so students can meet Assignment 1 and the class task expectations.

Learning objectives

- Read and write text, CSV and JSON files.
- Build a command-line tool with argparse.
- Replace print with basic logging; push work to GitHub.

Hour 1: File handling

- open() with context managers; text vs binary; encodings.
- csv.DictReader, json.load/json.dump.
- Why not to hard-code paths: use arguments.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Automate parsing of authentication logs and writing a JSON summary.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Automate reading and updating a cloud configuration JSON file.

Hour 2: Command-line tools

- argparse: positional arguments, options, --help, defaults, choices.
- Exit codes: sys.exit(0) for success and non-zero for failure; this matters for Bash and CI later.

```
import argparse, sys
p = argparse.ArgumentParser(description="Summarise a log file")
p.add_argument("path")
p.add_argument("--top", type=int, default=5, help="how many items to show")
args = p.parse_args()
print(f"Reading {args.path}, top {args.top}")
```

Hour 3: Practical and Git

- Lab: finish the CLI tool; add --output to write the summary to a file.
- Logging primer: logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s"); use logging.warning for skipped lines.
- Git: git add, git commit, git remote add origin, git push. Every student pushes their lab.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalization.

1. Run `python tool.py --help` and paste the exact output.
2. Run the tool with `--top` set to your seed (last digit of your roll number, minimum 3). Paste output and command.
3. Paste the log lines your tool produced at WARNING level for malformed input.
4. Share the URL of your GitHub repository and paste `git log --oneline` showing at least three commits made this week.
5. Run the tool on a non-existent file and paste what happens. Describe the change needed so that it exits with code 1 and a clear message.

Week 4: Bash Scripting Fundamentals

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Garrels, Bash Guide for Beginners, Ch. 1-3	Directory operations and cleanup script

Learning objectives

- Write Bash scripts with variables, conditionals, loops and arguments.
- Combine tools with pipes (grep, awk, sort, uniq, cut).
- Schedule a task with cron.

Hour 1: Bash basics

- Shebang, chmod +x, variables, quoting ("$var" always), $1, $#.
- Conditionals: if [-f "$file"], [[...]]; loops: for f in *.log.
- Always quote variables and test on a copy of the data.

> **Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)**
> Directory layout for evidence: case/YYYYMMDD/{raw,working,reports}. Script it.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> Organize a folder of exported cloud logs by date and region.

Hour 2: Pipes and system-level automation

- grep "Failed" auth.log | awk '{print $11}' | sort | uniq -c | sort -nr | head
- Redirection: >, >>, 2>&1.
- cron: the five fields, crontab -e, logging cron output to a file. Windows students use WSL cron or Task Scheduler.

Hour 3: Lab

- Write cleanup.sh: take a directory and an age in days, list files older than that, ask for confirmation, then delete. Default to a dry run.
- Schedule the dry run every hour and verify it fires once.

```
#!/usr/bin/env bash
dir="$1"; days="${2:-7}"
[ -d "$dir" ] || { echo "Not a directory: $dir" >&2; exit 1; }
find "$dir" -type f -mtime +"$days" -print
```

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalization.

1. Paste the output of your dry-run on your own test folder, showing the file list.

2. Run with a non-existent directory and paste the message and the exit code (`echo $?`).
3. Show your crontab line (or Task Scheduler screenshot) with the schedule expression, and the first log line proving it fired. Include the system clock.
4. Write a one-line pipe that reports the three most frequent values in a column of a file you created. Paste the command and its output.
5. Explain, using one real command from your script, why quoting `"$dir"` matters when a folder name contains a space. Demonstrate it.

Week 5: Bash for Backups, Evidence Handling and Cloud CLI

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Garrels, Bash Guide for Beginners, Ch. 5-6	Backup with cron and hashing

Revised for Fall 2026

Cloud CLI without a cloud account: use the AWS CLI with --endpoint-url http://localhost:5000 pointing to Moto (or another emulator). No real account or card is needed.

New: **Bash safety** (set -euo pipefail, trap) is the Bash equivalent of the exception primer.

Learning objectives

- Write Bash functions and handle errors with exit codes and trap.
- Automate backup with integrity verification.
- Use a cloud CLI against a local endpoint.

Hour 1: Advanced Bash

- Functions, return, local.
- set -euo pipefail and why it fails fast; trap cleanup EXIT.
- Logging from Bash with timestamps: echo "$(date -u +%FT%TZ) message" >> run.log.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Case study: evidence backup that copies, then verifies with sha256sum -c.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Case study: VM snapshot automation that must not run twice at once (lock file).

Hour 2: Integration with cloud CLI

- AWS CLI / Azure CLI install and --endpoint-url.
- Forensics: upload or sync a verified backup to object storage and record the hash.
- Never put access keys in scripts; use environment variables or a credentials profile.

```
aws --endpoint-url http://localhost:5000 s3 mb s3://lab-evidence
aws --endpoint-url http://localhost:5000 s3 cp backup.tar.gz s3://lab-evidence/
```

Hour 3: Lab

- Build backup.sh: tar a folder, write SHA256SUMS, verify, copy to a destination and log each step.
- Schedule it with cron every six hours (or every 5 minutes for the demonstration).

In-class / take-home questions (evidence-based)

Evidence rule

Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Paste your full backup.sh and one complete run log with timestamps.
2. Corrupt the backup file by one byte and run verification. Paste the failure output and exit code.
3. Show your crontab entry and the log lines from two consecutive scheduled runs.
4. Show a trap or error branch in your script and the exact line it printed when you caused a failure on purpose.
5. List the object(s) in your storage bucket (CLI output with the endpoint URL) and the SHA-256 recorded for the uploaded file.

Week 6: Web Scraping and Threat/Status Data Feeds

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Sweigart, Automate the Boring Stuff, Ch. 12	Scrape to CSV

Learning objectives

- Fetch and parse HTML with requests and BeautifulSoup.
- Scrape ethically: robots.txt, terms of use, rate limiting, identification.
- Store results in CSV.

Hour 1: Web scraping concepts

- HTTP GET, HTML structure, CSS selectors, soup.select().
- Prefer a documented feed or API over scraping a page when one exists.
- **Ethics and law:** check robots.txt and the terms; send a clear User-Agent; sleep between requests; never scrape personal data.

> **Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)**
> Targets: public threat-intelligence feeds that permit automated access (check the terms), or a static test page the instructor hosts.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> Targets: cloud-provider status pages and RSS/JSON status feeds, or an instructor-hosted test page.

Hour 2: Data storage

- Write rows with csv.DictWriter; include a collected_at UTC column and the source URL.
- Cloud track: write the CSV to a bucket on the emulator, or insert into SQLite as a stand-in for a cloud database.

Hour 3: Lab

- Scrape a page or feed, extract at least three fields, store them in CSV with timestamps.
- Handle a missing element without crashing.

```
import requests, csv, datetime
from bs4 import BeautifulSoup
r = requests.get(url, headers={"User-Agent": "HamdardLab/1.0"}, timeout=10)
r.raise_for_status()
soup = BeautifulSoup(r.text, "html.parser")
rows = [{"title": a.get_text(strip=True), "href": a.get("href")} for a in
soup.select("a")]
```

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalization.

1. State the exact URL scraped, paste the first five rows of your CSV, and show the robots.txt line you checked.
2. Show the collected_at value of your first row and your system clock at that moment.
3. Make one selector fail on purpose and paste how your script behaved.
4. Show the line of code that rate-limits your requests and explain the interval you chose.
5. Why is an official API or feed preferred over scraping HTML for the same data? Answer using a specific difference you saw in your own output.

Week 7: Node.js Basics for Scripting

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	Casciaro, Node.js Design Patterns, Ch. 1-2	File-renaming automation

Revised for Fall 2026

Use the current Node.js LTS. Teach fs/promises with async/await first; callbacks are covered as legacy (they come back in Week 12 as error-first callbacks).

Git thread: students create a feature branch for the lab and merge it with a pull request.

Learning objectives

- Run Node scripts, use modules and npm.
- Use fs/promises and path to rename and organise files.
- Understand callbacks, promises and async/await.

Hour 1: JavaScript and Node fundamentals

- const/let, arrow functions, template strings, destructuring, require vs import.
- process.argv for arguments; npm init -y; package.json.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Case study: rename evidence files to CASE-0001_YYYYMMDD_hash8.ext so they sort and cannot collide.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Case study: normalise object names before upload to storage.

Hour 2: Asynchronous programming

- Why Node is non-blocking; the event loop in one diagram.
- Callbacks, promises (.then), async/await with try/catch.
- Pitfall: forEach with await does not wait; use for...of or Promise.all.

```
const fs = require("fs/promises");
const path = require("path");
async function main(dir) {
  const names = await fs.readdir(dir);
  for (const [i, name] of names.entries()) {
    const ext = path.extname(name);
    const next = `file_${String(i + 1).padStart(4, "0")}${ext}`;
    console.log(`DRY RUN ${name} -> ${next}`);
  }
}
main(process.argv[2]).catch(e => { console.error(e.message); process.exit(1); });
```

Hour 3: Lab

- Add a --apply flag: dry run by default, rename only with the flag.
- Never overwrite: check that the target does not exist.

- Work on a branch, open a pull request, have a classmate review.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalisation.

1. Paste the dry-run output on your own test folder, then the folder listing after --apply.
2. Show how your script prevents overwriting a file and the message it prints when it refuses.
3. Show the pull-request URL for your lab and one review comment from a classmate.
4. Explain, using one line of your script, why await is needed there. What printed if you removed it?
5. Compare your Node.js renamer with how you would write it in Python. State one difference you actually observed.

Week 8: Node.js Scripting for Automation and Serverless Deployment

Mapped CLO	Suggested readings	Lab / activity
CLO 2 (Create)	Casciaro, Node.js Design Patterns, Ch. 3-4	Serverless function (local-first)

Revised for Fall 2026

The original lab requires a real AWS Lambda or Azure Functions account. That means card details and a billing risk for students. Use the tiers below and let each student pick the highest tier they can run safely.

Free-tier terms of public clouds change often. Verify the current AWS Free Tier / Azure free account terms yourself before offering Tier 4.

Learning objectives

- Explain serverless (functions as a service), events, cold starts and statelessness.
- Package a Node.js function and invoke it with an event.
- Deploy to a local or emulated serverless runtime.

Hour 1: Serverless concepts

- Event-driven functions, pay-per-invocation, stateless execution, time and memory limits.
- Trade-offs vs containers and VMs: scaling and cost vs cold start, vendor lock-in and debugging difficulty.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Case study: an on-upload function that hashes a new evidence file and writes the hash to a log.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Case study: an on-upload function that thumbnails an image or validates a JSON object.

Hour 2: Packaging and deployment

- Handler signature exports.handler = async (event) => {...}; returning a response object.
- Dependencies: npm ci --omit=dev, zip the folder, keep the package small.
- Secrets: environment variables, never in the zip.

```
exports.handler = async (event) => {
  const text = event.text ?? "";
  const words = text.split(/\s+/).filter(Boolean).length;
  return { statusCode: 200, body: JSON.stringify({ words }) };
};
```

Hour 3: Lab (choose a tier)

- **Tier 1, required for all:** run the handler locally with a tiny harness: node -e "require('./index').handler({text:'a b c'}).then(console.log)".

- **Tier 2:** deploy and invoke the zip on a local AWS emulator (Moto server with Docker available for Lambda execution, or another emulator).
- **Tier 3:** Azure Functions Core Tools (func start) running a Node.js HTTP function locally.
- **Tier 4, optional:** deploy to a real AWS or Azure free account if the student already has one and understands the billing alerts. Never required for marks.

In-class / take-home questions (evidence-based)

Evidence rule

Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Paste the exact output of your Tier 1 harness for three different inputs, including an empty one.
2. State the tier you completed, and paste the invoke command and the JSON response.
3. Add input validation that returns status 400 for non-string input. Paste the code and the response for a bad event.
4. Report the time (ms) of the first and second invocation, and explain any difference in terms of cold starts or process start-up.
5. Name one workload in your own track that is a good fit for serverless and one that is not. Justify both in one sentence each.

Suggested date for the one-hour class task (mid-semester)

The approved plan says "mid-semester, Weeks 1-8" without a date. Run it at the end of the Week 8 lab slot (after the local-first serverless lab, which is shorter than the original cloud-account version). That keeps it before the Week 9 midterm. Every skill it tests (exceptions, logging, Bash exit codes, Git) now has a primer by Week 5.

Part B: Weeks 9-16 (APIs, Reliability, Containers, CI/CD)

Weeks 10-15 follow the original notes (objectives, three hours, step-by-step method, evidence-based questions) with the Fall 2026 corrections. They form one continuous project: the **Week 10 fetch script** becomes the **Week 11 workflow**, is hardened in **Weeks 12-13**, containerized in **Week 14**, and put through a pipeline in **Week 15**. Students should keep a single repository for all of it.

Week 9: Midterm Examination

Mapped CLO	Coverage	Marks
CLO 1 (12) and CLO 2 (8)	Weeks 1-8	20

No new content. Use the first hour for a consolidated revision session on Python syntax, file handling, argparse, Bash automation, scraping, Node.js basics and serverless concepts. The remaining two hours are the paper. The two outlines describe the midterm as "theory only", yet the original notes also mention a "written/practical" format. I recommend a **closed-book written paper with code-reading and short code-writing questions**, which fits "theory only" but still tests CLO 1 properly.

Suggested paper blueprint (20 marks)

Q	Type	Content	Marks	CLO
1	Short answers (4 x 1)	Scripting vs programming; Python types; Bash quoting; cron fields	4	1
2	Code reading	Trace output of a Python loop/dict snippet and a Bash one-liner	4	1
3	Code writing	Write a function that parses a log/CSV line, skipping malformed ones	4	1
4	Design	Design a Bash + cron backup with integrity check (steps, commands, failure behaviour)	4	2
5	Scenario	Choose and justify serverless vs VM vs container for a described workload	4	2
	Total		**20**	CLO 1 = 12, CLO 2 = 8

Week 10: API Interaction: REST APIs and JSON Parsing

Mapped CLO	Suggested readings	Lab / activity
CLO 1 (Apply)	ATBS (2nd ed.) Ch. 16 (CSV and JSON); Python requests Quickstart; HTTP status codes (MDN)	Fetch, parse, save and hash evidence

Revised for Fall 2026

Reading changed: *Practical Malware Analysis* Ch. 6 is about x86 control flow, not APIs.

Cloud track: **LocalStack Community is no longer freely available**; use Moto server (or Floci / LocalStack with a free token).

datetime.utcnow() is deprecated; the scripts below use timezone-aware UTC.

Node.js parity added because Assignment 2 is a Node.js API task.

Fallback **mock API** added so the lab works even if a free-tier API is down or rate-limited.

Learning objectives

- Explain REST principles (resources, verbs, statelessness) and HTTP status code classes.
- Parse nested JSON safely with dict.get, try/except KeyError and type checks.
- Build authenticated, time-limited, error-tolerant requests with Python requests and Node.js fetch.
- Persist responses as timestamped files with a SHA-256 hash and an audit log entry.

Hour 1: REST, HTTP and status codes

- An API is a contract between programs. REST uses URLs for resources and verbs: GET reads, POST creates, PUT/PATCH updates, DELETE removes.
- Status classes: 2xx success; 4xx client problem (401 unauthenticated, 403 forbidden, 404 not found, 429 rate limit); 5xx server failure. A script that treats a 403 as data corrupts the record.
- Credentials travel in headers (API key, bearer token). They must never be hard-coded or committed. Store them in .env and add .env to .gitignore on day one.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Platforms such as VirusTotal, AbuseIPDB and Shodan expose reputation data over REST. Scripting the call gives a reproducible, timestamped record, which supports chain of custody.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

AWS, Azure and GCP expose every service over REST beneath the SDKs. A 403 usually means an IAM permission problem; a script that carries on would act on a false assumption.

Hour 2: JSON as the data-interchange format

- JSON objects map to Python dicts and arrays to lists; response.json() or json.loads() converts text to Python objects.
- Real responses are nested and vary between vendors and versions: use data.get("data", {}).get("abuseConfidenceScore") or catch KeyError.

- Check types before comparing (is the score really an int?). Schemas change silently.
- Why JSON and not a flat CSV row? Nested objects and arrays (list of detections, list of resources) do not fit one row without losing structure.

Hour 3: Python workflow with requests

- Always pass timeout=; check status_code before parsing; catch requests.exceptions.RequestException.
- Save the raw JSON immediately with a UTC timestamp in the file name ("evidence at rest"), compute its SHA-256, and append a line to a custody/audit log.
- Node.js parity: Node 18+ has built-in fetch; the same discipline applies.

```
import os, json, hashlib, datetime, logging
import requests
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("API_KEY")
URL = os.getenv("API_URL", "https://api.abuseipdb.com/api/v2/check")

def utc_stamp():
    return datetime.datetime.now(datetime.timezone.utc).strftime("%Y%m%d_%H%M%S")

def fetch_ip(ip):
    headers = {"Key": API_KEY, "Accept": "application/json"}
    params = {"ipAddress": ip, "maxAgeInDays": 90}
    resp = requests.get(URL, headers=headers, params=params, timeout=10)
    if resp.status_code != 200:
        raise RuntimeError(f"HTTP {resp.status_code}: {resp.text[:200]}")
    return resp.json()

def save_evidence(data, url):
    ts = utc_stamp()
    fname = f"evidence_{ts}.json"
    with open(fname, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    with open(fname, "rb") as f:
        digest = hashlib.sha256(f.read()).hexdigest()
    with open("custody.log", "a", encoding="utf-8") as log:
        log.write(f"{ts}\t{url}\t{fname}\tSHA256={digest}\n")
    return fname, digest

if __name__ == "__main__":
    try:
        data = fetch_ip("203.0.113.9")
        print(save_evidence(data, URL))
        print("score:", data.get("data", {}).get("abuseConfidenceScore"))
    except requests.exceptions.RequestException as e:
        print("Network failure:", e)
    except RuntimeError as e:
        print("API failure:", e)
```

Node.js equivalent (for Assignment 2):

```
const fs = require("fs/promises");
require("dotenv").config();

async function main() {
  try {
    const res = await fetch(process.env.API_URL, {
      headers: { Key: process.env.API_KEY, Accept: "application/json" },
      signal: AbortSignal.timeout(10000),
    });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    const ts = new Date().toISOString().replace(/[-:]/g, "").slice(0, 15);
    await fs.writeFile(`evidence_${ts}.json`, JSON.stringify(data, null, 2));
    console.log("saved", ts);
  } catch (err) {
    console.error("Fetch failed:", err.message);
    process.exitCode = 1;
  }
}
main();
```

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Query a free-tier threat-intelligence API for one IP address (use documentation or test addresses such as 203.0.113.0/24 where the provider allows). Free-tier daily limits and header names change; check the provider's current documentation before the lab.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Start a local emulator and query it as you would a real endpoint. With Moto: pip install "moto[server]" then moto_server -p 5000. Create a bucket with boto3 (dummy credentials test/test, endpoint_url="http://localhost:5000"), then call requests.get(BASE + STATE_PATH) where STATE_PATH is /moto-api/data.json (confirm on your installed version). If you use LocalStack with a free token, the health path is /_localstack/health. Keep CLOUD_URL and STATE_PATH in .env so the same script runs on any emulator.

Instructor fallback: 25-line mock API

Run this on the instructor PC or on each student machine (python mock_api.py). It returns 401 for a wrong key, 200 for the first three valid calls and 429 afterwards. Restart it to reset the counter. Point API_URL at http://127.0.0.1:8000/ and use the key lab-key.

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json, datetime
calls = 0
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        global calls
        if self.headers.get("Key") != "lab-key":
            return self._send(401, {"errors": [{"detail": "Authentication failed"}]})
        calls += 1
        if calls > 3:
            return self._send(429, {"errors": [{"detail": "Rate limit exceeded"}]})
        now = datetime.datetime.now(datetime.timezone.utc).isoformat()
        self._send(200, {"data": {"ipAddress": "203.0.113.9", "abuseConfidenceScore":
77,
                                  "totalReports": 12, "lastReportedAt": now}})
```

```
    def _send(self, code, body):
        b = json.dumps(body).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(b)))
        self.end_headers()
        self.wfile.write(b)
HTTPServer(("127.0.0.1", 8000), H).serve_forever()
```

Step-by-step lab method (free and open-source tools only)

Tools: Python 3.12+, VS Code, requests, python-dotenv; Node.js LTS; Forensic: any free-tier threat-intelligence API or the mock; Cloud: Moto server (or Floci / LocalStack with free token).

1. Create a project folder and a virtual environment (see command table in Section 3), then pip install requests python-dotenv.
2. Create .env with API_KEY=... and API_URL=... and add .env to .gitignore **before** the first commit.
3. Write api_lookup.py as above. Run it and confirm a 200 response.
4. Open the saved JSON and identify the nested structure: list the top-level keys and one nested field.
5. Record the SHA-256 of the saved file and compare it with the hash in custody.log.
6. Re-run with an invalid key (or stopped emulator) and record the status code or connection error.
7. Run three rapid requests (mock API: observe the 429 on the fourth call).
8. Write the Node.js version and save a second timestamped file.
9. Commit the code (not the evidence, not .env) and push.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Execute a live GET request from your machine and report the exact HTTP status code and the first five top-level keys of the returned JSON. Include a screenshot showing the system clock.
2. **Forensic:** send an invalid API key and paste the exact status code and JSON error message. **Cloud:** stop the emulator and paste the exact connection error.
3. From the response you retrieved, extract one nested field of your choosing. Show the single line of Python used and the value.
4. Using the file you saved, explain why a nested JSON object rather than a flat CSV row was necessary. Refer to the real structure you observed.
5. Compute the SHA-256 of the saved JSON file and paste the exact terminal output of the command.
6. Send four rapid requests to the mock API (or three to the real one). Report the status code of each call and explain what safeguard (or limit) produced it.
7. Show your .env structure with the key redacted, the Python lines that load it, and one sentence on why hard-coding a credential is a security risk. Show git status proving .env is ignored.

8. Compare the timestamp inside the JSON payload (if present) with your system clock at the time of the request. Explain any difference you observed (time zone, clock skew, caching).

Week 11: Automating Workflows with APIs

Mapped CLO	Suggested readings	Lab / activity
CLO 2 (Create)	ATBS (2nd ed.) Ch. 17-18 (scheduling, email); Python sqlite3 and smtplib docs	Fetch, evaluate, alert; SQLite state; scheduler

Revised for Fall 2026

Reading changed: Ch. 14 is Google Sheets in the 2nd edition; verify numbering for your edition.

Primary alert channel is a **free webhook** (Discord/Slack). A local aiosmtpd debug SMTP server gives an offline email demonstration. The standard-library smtpd module was removed in Python 3.12.

Windows students use Task Scheduler instead of cron (see Section 3).

Learning objectives

- Chain API calls into a fetch-parse-evaluate-act workflow.
- Write small, testable functions with defensive validation.
- Persist state in SQLite to prevent duplicate alerts.
- Run the workflow unattended with a system scheduler and reason about idempotency.

Hour 1: From single script to workflow

- A script answers one question once; a workflow decides and acts repeatedly. This synthesis is the "Create" level (CLO 2).
- Shape: fetch, parse, evaluate, act. Keep each stage a function so each can be tested alone.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Use case: alert the team when a monitored IP's abuse confidence rises above a threshold since the last check.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Use case: alert when a storage bucket's object count or a queue backlog crosses a threshold.

Hour 2: Building the multi-step automation

- fetch_data() (reuse Week 10), evaluate(data) returns True/False after validating that the field exists and is numeric, send_alert(text) performs the action.
- Alert channels: webhook POST with requests (preferred); email with smtplib and an app password; or a local aiosmtpd server for demonstration.
- Never assume the field exists: a malformed response must not crash an unattended job overnight.

```
import os, sqlite3, datetime, logging, requests
THRESHOLD = int(os.getenv("THRESHOLD", "50"))

def evaluate(data):
    try:
        score = data["data"]["abuseConfidenceScore"]
    except (KeyError, TypeError) as e:
        raise ValueError(f"missing field: {e}") from e
    if isinstance(score, bool) or not isinstance(score, (int, float)):
        raise ValueError(f"score not numeric: {score!r}")
```

```
    return score > THRESHOLD

def send_alert(text):
    r = requests.post(os.environ["WEBHOOK_URL"], json={"content": text}, timeout=10)
    r.raise_for_status()
```

Hour 3: State persistence and unattended scheduling

- SQLite (built in) stores what has already been alerted on: entity, timestamp, action.
- Check the table before alerting; insert a row **only after** a successful send.
- Schedule with cron (*/30 * * * *) or Task Scheduler. Use absolute paths and log to a file; cron has a minimal environment.
- **Idempotency:** running twice for the same underlying data must produce one alert, not two.

```
def already_alerted(conn, entity, hours=24):
    cutoff = (datetime.datetime.now(datetime.timezone.utc) -
datetime.timedelta(hours=hours)).isoformat()
    return conn.execute("SELECT 1 FROM alerts WHERE entity=? AND ts>? LIMIT 1",
(entity, cutoff)).fetchone() is not None

conn = sqlite3.connect("alerts.db")
conn.execute("CREATE TABLE IF NOT EXISTS alerts (entity TEXT, ts TEXT, action TEXT)")
# ... after a successful send_alert():
conn.execute("INSERT INTO alerts VALUES (?,?,?)", (entity,
datetime.datetime.now(datetime.timezone.utc).isoformat(), "webhook"))
conn.commit()
```

Optional local email demo: run python -m aiosmtpd -n -l localhost:1025 in one terminal (it prints each message it receives), then send with smtplib.SMTP("localhost", 1025) from your script. No account is needed.

Step-by-step lab method (free and open-source tools only)

Tools: Python 3.12+, requests, built-in sqlite3, a free Discord/Slack webhook (or aiosmtpd), cron or Task Scheduler. Cloud track: the Week 10 emulator is the data source.

1. Reuse fetch_data() from Week 10 as the retrieval stage.
2. Write evaluate(data) with validation and a configurable threshold (use your personal seed).
3. Create the SQLite database and alerts table.
4. Before alerting, check already_alerted() for this entity.
5. Write send_alert() using a webhook or local SMTP.
6. Wrap main() in try/except that logs a failure instead of crashing.
7. Insert a row into alerts only after a successful send.
8. Schedule: Linux crontab -e with */30 * * * * /usr/bin/python3 /path/to/workflow.py; Windows Task Scheduler (repeat every 30 minutes).
9. After the first scheduled run, inspect the state: sqlite3 alerts.db "SELECT * FROM alerts;".

In-class / take-home questions (evidence-based)

Evidence rule

Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Run the complete fetch-evaluate-act workflow and paste the exact terminal output of one pass, showing the sequence of function calls.
2. Trigger the True branch of evaluate() using live (or mock) data and show the exact alert message produced.
3. Query alerts.db after your first scheduled run and paste the exact rows.
4. Using your own table contents, explain how the stored state prevented a duplicate alert on the second run.
5. Show your crontab entry (or Task Scheduler screenshot) including the schedule expression.
6. Corrupt one field in a test JSON response and report the exact exception your validation caught.
7. Report the actual wall-clock time your scheduled job fired versus the configured time, and explain any drift.
8. Identify one point where duplicate side effects could still occur (for example, the alert is sent but the process dies before the INSERT) and describe the specific code change that closes the gap.

Week 12: Error Handling, Debugging and Collaboration

Mapped CLO	Suggested readings	Lab / activity
CLO 3 (Analyze)	Lutz, Learning Python, Ch. 10 (or the Python tutorial chapter on Errors and Exceptions); Pro Git, Ch. 1-3 (free online)	pdb, custom exceptions, Git pull-request workflow

Revised for Fall 2026

Gap closed: CLO 3 explicitly requires version control and collaborative project management, but the original notes had no Git content. Hour 3 now includes a pull-request workflow for the debugging lab.

Node.js section keeps async/await as the primary model; error-first callbacks are taught as legacy.

Learning objectives

- Distinguish syntax, runtime and logical errors and choose a strategy for each.
- Use try/except/else/finally and custom exceptions.
- Debug with pdb and VS Code breakpoints and read tracebacks bottom to top.
- Collaborate with branches, pull requests and code review.

Hour 1: A taxonomy of errors

- **Syntax errors** are caught before the program runs. **Runtime errors** (exceptions) occur during execution: missing file, timeout, missing key. **Logical errors** are the most dangerous: the script finishes but the result is wrong.
- try contains the risky code; catch the most specific exception first; else runs if nothing failed; finally always runs (close files, release locks).
- Custom exceptions make failure modes self-documenting.

```
class EvidenceIntegrityError(Exception):
    """Raised when a computed hash does not match the recorded hash."""

def verify(path, expected):
    actual = sha256_of(path)
    if actual != expected:
        raise EvidenceIntegrityError(f"{path}: expected {expected[:12]}..., got
{actual[:12]}...")
```

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Domain exception: EvidenceIntegrityError when a hash does not match.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Domain exception: ResourceProvisioningError when the cloud API confirms a resource was not created.

Hour 2: Debugging beyond print()

- python3 -m pdb script.py; n next, s step into, c continue, b 42 breakpoint, p var print, l list, q quit.
- Reading a traceback: the exception type and message are at the **bottom**; the innermost frame is just above it.
- Never write except: pass. Every I/O, hash and ingestion point needs an explicit, logged failure path.
- Log the full stack with logging.error(traceback.format_exc()) or logging.exception(...).

> **Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)**
> A silent failure in an evidence script can produce an incomplete report that nobody notices.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> A silent failure in a provisioning script can leave infrastructure in an inconsistent or costly state.

Hour 3: Node.js error handling and team workflow

- Node: try/catch/finally with async/await; .catch() on promises; error-first callbacks in older APIs (cb(err, data)).
- **Git collaboration lab:** create a branch fix/evaluate-bug, commit the fix, push, open a **pull request**, request a review from a classmate, address one comment, and merge. Write a clear commit message and PR description.
- Agree a team convention: all scripts, regardless of language, log errors in the same structured format (this prepares Week 13).

```
async function run(url) {
  try {
    const res = await fetch(url, { signal: AbortSignal.timeout(10000) });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("Fetch failed:", err.message);
    throw err;
  } finally {
    console.log("fetch attempt finished");
  }
}
```

Step-by-step lab method (free and open-source tools only)

Tools: Python pdb and traceback, VS Code debugger, Node.js, Git and GitHub (all free).

1. Introduce a deliberate bug into your Week 11 evaluate() (for example misspell a variable).
2. Launch python3 -m pdb workflow.py; use n, c, p to locate the line.
3. Fix the bug on a Git branch.
4. Wrap the relevant I/O in a specific handler: FileNotFoundError (Forensic) or requests.exceptions.RequestException with raise_for_status() (Cloud).
5. Define and raise one custom exception.
6. Reproduce the logic error in a small Node.js script and catch it with try/catch.
7. Add logging.error(traceback.format_exc()).

8. Re-run end to end and confirm graceful degradation.
9. Set a breakpoint in VS Code and see the same root cause.
10. Open a pull request for the fix and get one peer review.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Reproduce a runtime error in pdb and paste the exact session transcript (commands and output).
2. Paste the full traceback produced before your fix and explain which line was actually at fault, reading from bottom to top.
3. Show your custom exception class and one real line where it is raised, with the triggering condition.
4. **Forensic:** rename or delete the evidence file and paste the exact log line from your FileNotFoundError handler. **Cloud:** stop the emulator and paste the exact log line from your RequestException handler.
5. In Node.js, cause the awaited call to fail (invalid URL) and paste the exact console.error output.
6. Paste the traceback.format_exc() output captured in your log file for one real exception.
7. Set a breakpoint at the faulty line in VS Code and describe the exact variable values you saw.
8. Find one place where a broad except: pass could hide a real failure and show the specific handler you would use instead.
9. Give the URL of your pull request, the review comment you received and the commit that resolved it.

Week 13: Logging and Monitoring

Mapped CLO	Suggested readings	Lab / activity
CLO 3 (Analyze)	Python "Logging HOWTO" and "Logging Cookbook" (docs.python.org); Winston README; Grafana Loki documentation	Python logging, Winston, optional Loki

Revised for Fall 2026

Reading changed: *Docker Deep Dive* Ch. 2-3 does not cover logging; move it to Week 14 where it fits.

Promtail is deprecated in favour of Grafana Alloy (support ended in early 2026). The Cloud track step that "points Promtail at the log file" should use **Grafana Alloy**, or the simpler fallback below (push lines to Loki's HTTP API). Check Grafana's current docs for the configuration syntax.

The original basicConfig(filename=...) and RotatingFileHandler snippets conflicted. A single correct configuration is given below.

The "hash at start and end" exercise is revised: a log that is still being written changes by definition. Hash **closed (rotated) files** and store the hashes elsewhere.

Learning objectives

- Explain why persistent, timestamped, severity-tagged logs differ from print debugging.
- Configure Python logging with console, rotating file and JSON handlers.
- Implement equivalent logging in Node.js with Winston.
- Describe how logs are aggregated and how tampering can be detected.

Hour 1: Why logging matters

- A log is persistent process evidence: it outlives the run and is read by someone else (an auditor, an on-call engineer).
- Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL. INFO for normal operation, ERROR for real failures; misusing levels makes logs useless.
- Timestamps in UTC with timezone, consistent format, no secrets in logs.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Admissibility theme: you must be able to show what the tool did and when, and that the record was not altered afterwards.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Operations theme: an on-call engineer diagnoses an incident hours later using only the log.

Hour 2: Structured logging in Python

- Replace every print() with the right level.
- RotatingFileHandler caps size and keeps backups, so a long-running monitor does not fill the disk.
- Emit JSON lines so logs can be queried without fragile text parsing.

```
import logging, json, datetime
from logging.handlers import RotatingFileHandler

class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({
            "ts": datetime.datetime.fromtimestamp(r.created,
datetime.timezone.utc).isoformat(),
            "level": r.levelname, "msg": r.getMessage(),
            "logger": r.name, "context": getattr(r, "context", {}),
        })

log = logging.getLogger("workflow")
log.setLevel(logging.INFO)
text = RotatingFileHandler("workflow.log", maxBytes=5_000_000, backupCount=3,
encoding="utf-8")
text.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
jsonl = RotatingFileHandler("workflow.jsonl", maxBytes=5_000_000, backupCount=3,
encoding="utf-8")
jsonl.setFormatter(JsonFormatter())
console = logging.StreamHandler()
console.setLevel(logging.WARNING)
for hd in (text, jsonl, console):
    log.addHandler(hd)

log.info("fetched", extra={"context": {"entity": "203.0.113.9", "score": 77}})
```

Hour 3: Winston and an open-source monitoring stack

- Winston: levels, transports (console and file together), formats.
- Grafana Loki + Grafana are free, self-hosted log aggregation and search.
- Tamper evidence: store the hash of each **closed** log file in a separate append-only record or a different system. Optional extension: a hash chain where each line carries the hash of the previous line.

```
const winston = require("winston");
const logger = winston.createLogger({
  level: "info",
  format: winston.format.combine(winston.format.timestamp(), winston.format.json()),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: "workflow.log" }),
  ],
});
logger.error("fetch failed", { url, err: err.message });
```

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Fallback if Alloy is unavailable: start Loki with `docker run -d -p 3100:3100 grafana/loki` and Grafana with `docker run -d -p 3000:3000 grafana/grafana`, then push a line from Python with a POST to `http://localhost:3100/loki/api/v1/push` using a JSON body `{"streams":[{"stream":{"job":"lab"},"values":[["<unix-ns-timestamp>","<line>"]]}]}`. Add Loki as a data source in Grafana (URL `http://host.docker.internal:3100`, or a shared Docker network) and query `{job="lab"}` in Explore.

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)
Loki/Grafana is optional for this track. The required tamper-evidence exercise is the hash of a rotated log file recorded in a separate file, then re-verified.

Step-by-step lab method (free and open-source tools only)

Tools: Python logging, Winston (npm install winston), sha256sum or Get-FileHash; optional Docker with Loki and Grafana.

1. Add the logging configuration above to your Week 12 script.
2. Replace all remaining print() calls with the correct level.
3. Set maxBytes=2000 temporarily and run until the file rotates; then restore 5 MB.
4. Add Winston to a Node.js script with console and file transports.
5. Trigger one deliberate error in each script and confirm it appears at ERROR level in both logs.
6. Hash a rotated (closed) log file, store the hash in hashes.txt, change one character in a copy, and re-verify.
7. Add a retention-policy comment at the top of each script (how long logs are kept and why).
8. **Cloud:** run Loki and Grafana and view your logs in Explore. **Forensic:** optional.

In-class / take-home questions (evidence-based)

Evidence rule
Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalisation.

1. Paste the exact log lines your script produced for one INFO and one ERROR event, including the timestamp.
2. Trigger rotation (set maxBytes to 2000) and list the files that appeared. If you cannot, calculate how many average-length lines fit in 5,000,000 bytes using a line length you measured.
3. Paste the console and file output Winston produced for one deliberate error.
4. Hash a rotated log file and paste the SHA-256. Then change one character in a copy and paste its hash. Explain what a verifier would conclude.
5. Show one real JSON log entry and the exact Python code that produced it.
6. **Cloud:** screenshot of your log stream in Grafana Explore with the query used. **Forensic:** the exact docker run command you would use to start Loki, and the port it exposes.
7. Identify one log statement using the wrong severity and give the corrected line.
8. Explain, using your retention-policy comment, how a longer or shorter retention period would change the usefulness of this log (evidential for Forensic, diagnostic for Cloud).

Week 14: Introduction to Docker and Containerisation

Mapped CLO	Suggested readings	Lab / activity
CLO 2 (Create)	Poulton, Docker Deep Dive, Ch. 2-5; Docker "Dockerfile best practices"	Build and run a container for the workflow

Revised for Fall 2026

.dockerignore excludes .env, but the script needs its key at run time. Pass secrets at **run time** with --env-file or -e, never bake them into the image.

The image now runs as a **non-root user** and uses a pinned base tag.

Windows path syntax added for volume mounts.

Lab network tip: pre-pull python:3.12-slim and share it with docker save / docker load so 30 students do not hit Docker Hub limits at once.

Learning objectives

- Explain images, containers, registries and how containers differ from virtual machines.
- Write a minimal, cache-friendly, least-privilege Dockerfile for a Python script.
- Build, run, mount volumes and (optionally) push an image.

Hour 1: Why containerize

- "Works on my machine" arises from different Python versions, libraries and OS settings. An image packages the application with its exact dependencies.
- A VM virtualises hardware and runs its own kernel; a container shares the host kernel and isolates processes and file systems, so it starts in about a second.
- Core terms: **image** (read-only template), **container** (running instance), **registry** (Docker Hub, ECR).

Forensic track (DFSP-5208, BS Digital Forensics & Cyber Security)

Reproducibility: the same tool produces the same output on every investigator's machine.

Cloud track (CCSP-5213, BS Cloud Application Development & Operations)

Portability: the same image runs in development, staging and production.

Hour 2: Writing the Dockerfile

- Choose a slim official base (python:3.12-slim).
- Copy requirements.txt first and install, then copy the code. Docker caches layers, so code changes do not reinstall dependencies.
- .dockerignore keeps venv/, .env, .git, evidence and logs out of the image.
- Run as a non-root user. ENTRYPOINT makes the script behave like a tool.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser && mkdir -p /app/evidence && chown -R appuser /app
```

```
USER appuser
ENTRYPOINT ["python3", "workflow.py"]

# .dockerignore
venv/
.env
.git/
__pycache__/
*.log
evidence/
output/
```

Hour 3: Build, run, share

- docker build -t forensic-workflow . then docker run --rm --env-file .env -v "$(pwd)/evidence:/app/evidence" forensic-workflow (Windows PowerShell: ${PWD}).
- Without -v, the container file system is discarded when it stops, so evidence written inside is lost.
- If the mounted folder is not writable on Linux, add --user "$(id -u):$(id -g)".
- Compare the container output to the Week 11 host run; they should match.
- docker login, docker tag, docker push to a free Docker Hub repository (optional). Never push an image that contains secrets.

Step-by-step lab method (free and open-source tools only)

Tools: Docker Engine (Linux) or Docker Desktop (education/personal use, check current terms); optional free Docker Hub account.

1. Install Docker and verify with docker run --rm hello-world.
2. Create requirements.txt listing exactly the third-party packages used.
3. Write the Dockerfile and .dockerignore.
4. Build: docker build -t forensic-workflow . (Cloud: cloud-workflow).
5. Run with --env-file and the evidence/output volume.
6. Compare the container output with the host output from Week 11.
7. Check size with docker images.
8. Run docker run --rm --entrypoint sh <image> -c "ls -a /app" and confirm that .env is **not** inside.
9. (Optional) log in, tag and push to Docker Hub.

In-class / take-home questions (evidence-based)

> **Evidence rule**
> Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalized, see the Appendix on personalisation.

1. Paste the exact docker build output, including the final image ID.
2. Run docker images and report the exact size. Build once with python:3.12 (non-slim) and compare the sizes.
3. Run with the volume mount and paste the output confirming the evidence/output file appeared on the host.

4. Run once **without** -v, report what happened to the output file and explain in terms of the container file-system lifecycle.
5. Compare the container run with your Week 11 host run on the same input and report whether they matched exactly.
6. Show your .dockerignore and explain, with a real file, what would have gone wrong without it. Paste the output proving .env is not in the image.
7. If you did the push: paste the exact docker push output with the repository name.
8. Introduce an ordering mistake (copy all files before installing dependencies), change only application code, and report the rebuild time difference versus the correct order.

Week 15: Scripting for CI/CD Pipelines

Mapped CLO	Suggested readings	Lab / activity
CLO 2 (Create)	Erl, Cloud Computing: Concepts, Technology & Architecture, Ch. 8-9; GitHub Actions documentation; Trivy documentation	Test, build and scan pipeline

Revised for Fall 2026

aquasecurity/trivy-action@master is unpinned and the original step never fails the build. In a security course, **pin third-party actions to a released tag or a full commit SHA** and make the scan enforce a policy.

Add permissions: contents: read (least privilege).

Check the current major versions of actions/checkout and actions/setup-python and update the examples if newer ones exist.

Cloud track: an optional integration job runs a Moto emulator as a service container.

Git/CLO 3: protect main; merge only through a reviewed pull request.

Learning objectives

- Distinguish continuous integration from continuous deployment.
- Write a GitHub Actions workflow using the free tier.
- Assemble test, build and vulnerability-scan stages and enforce a pass/fail policy.

Hour 1: CI/CD concepts

- CI builds and tests every change automatically; CD ships changes that pass all checks.
- For security tooling, CI/CD gives repeatability and an auditable, timestamped record of what was tested and shipped.
- This week assembles everything: scripts (Weeks 1-11), hardening (12-13), image (14).

Hour 2: GitHub Actions

- Workflows live in .github/workflows/*.yml: name, triggers (on), jobs, steps; each job runs on a fresh runner.
- YAML is indentation-sensitive; learn to read the Actions log.
- Free for public repositories and with a monthly quota for private ones (check current limits).
- Treat third-party actions as dependencies: pin and review.

Hour 3: Test, build, scan

- Stage 1: install dependencies, py_compile smoke test and any unit tests.
- Stage 2: docker build with the commit SHA as tag.
- Stage 3: Trivy scans the image for known vulnerabilities and fails on HIGH/CRITICAL (ignore unfixed ones to keep the signal useful).
- Test must come before build: do not spend time building an image from code that does not even compile.

```yaml
name: CI
on:
  push:
  pull_request:
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python -m py_compile workflow.py
      - run: python -m unittest discover -s tests
  build-and-scan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t forensic-tool:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@<PINNED_TAG_OR_COMMIT_SHA>
        with:
          image-ref: forensic-tool:${{ github.sha }}
          format: table
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
```

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> Optional integration job: add services: moto: { image: motoserver/moto:latest, ports: ["5000:5000"] } and run your Week 10 script against http://localhost:5000. Pin the image tag in real projects.

Step-by-step lab method (free and open-source tools only)

Tools: Free GitHub account, GitHub Actions, Docker (Week 14), Trivy via its official GitHub Action.

1. Create a GitHub repository and push the Week 14 project (script, Dockerfile, requirements, tests).
2. Add .github/workflows/ci.yml as above; pin the Trivy action to a released tag you looked up today.
3. Add a minimal unit test for evaluate() (normal, missing field, wrong type).
4. Push and watch the Actions tab; read each step's log.
5. If the pipeline is red, read the failing step and fix it.
6. Turn on branch protection for main (require a pull request and a passing check).
7. Make one change through a pull request and note the new commit SHA and image tag.
8. Write a README that documents each stage and which CLO it demonstrates.

In-class / take-home questions (evidence-based)

Evidence rule

Every answer must be backed by output produced on the student's own machine this week (terminal output, screenshot showing the system clock, file hash or log). Generic, non-executed answers earn zero. Parameters are personalised, see the Appendix on personalisation.

1. Paste the exact YAML of your committed ci.yml.
2. Report the status (success/failure) of every step in your first run.
3. If any step failed, paste the exact log line that revealed the failure and the fix you made.
4. Report the exact image tag produced for your most recent successful run, with the SHA substituted.
5. Paste the Trivy summary output including the number of vulnerabilities found (even zero) and the severity filter you used.
6. Make a small change, push through a PR, and report the new commit SHA and the new image tag generated automatically.
7. Referring to your own step order, explain why the build comes after the test.
8. Quote the sentence from your README that says which CLO the Trivy stage demonstrates, and justify the mapping in one more sentence.
9. Show a pull request where the check failed, and the follow-up commit that made it pass.

Week 16: Best Practices and Final Project Presentations

Mapped CLO	Suggested readings	Lab / activity
CLO 2 and CLO 3	Consolidated readings; GitHub README guides	Capstone demonstration

Revised for Fall 2026

The original notes stopped at Week 15, but both outlines schedule Week 16 (final project presentation, capstone lab). This week is new.

Hour 1: Best practices checklist

- **Secrets:** no credentials in code, logs, images or commits. .env ignored; use --env-file at runtime.
- **Reliability:** timeouts on every network call, specific exceptions, no silent except: pass, idempotent scheduled jobs.
- **Observability:** structured logs at the right level, rotation, retention policy.
- **Reproducibility:** pinned versions, requirements.txt, Dockerfile, CI on every push.
- **Collaboration:** small commits, meaningful messages, pull requests with review, README with run instructions.
- **Ethics and law:** respect terms of service and robots.txt, lab data only, no scanning of systems you do not own or have permission to test.

Hours 2-3: Final project presentations

Each student or team (maximum three) presents a **10-minute demo and 5-minute viva** of the capstone from Assignment 3: a script from earlier weeks, now with logging, error handling, a Dockerfile and a CI workflow.

Component	What to show	Evidence
Application	What it does, for which track (forensic or cloud)	Live run, console output
Reliability	One handled failure demonstrated live	Log line and traceback
Container	Build and run, with a volume mount	docker build and run output
Pipeline	A green run and one deliberately failing run	Actions tab, Trivy summary
Collaboration	Commit history, a pull request and review	GitHub links
Viva (5 min)	Instructor asks about a line of code, a design choice and a failure mode	Spoken answers

Marks

The approved outlines give the final presentation no separate marks; its assessment value comes through Assignment 3 (4 marks) and the final exam. Use the presentation as the **viva check** that the submitted Assignment 3 is the student's own work. If you want presentation marks, they must come out of an approved component, so decide that before you announce it.

Final exam revision plan (Weeks 1-16)

Block	Revisit	Weeks
A: Scripting fundamentals (CLO 1)	Python, Bash and Node.js syntax; file parsing; CLI tools	1-8
B: Design and deployment (CLO 2)	API workflow design, serverless, Dockerfile ordering, CI/CD pipeline	8, 10, 11, 14, 15
C: Analysis and reliability (CLO 3)	Tracebacks, debugging, custom exceptions, logging levels and rotation, Git workflow	12, 13, 15

Part C: Assessment Pack (Fall 2026)

All instruments follow the approved marks distribution (Section 2). Questions are written so that a correct answer needs a **personal artefact** (own output, own hash, own repository), which also addresses the academic-integrity expectations in the outlines. Model answers are given for quick marking.

Quiz 1 (Weeks 1-5), 3 marks, closed book, 15 minutes

Q	Question	CLO	Marks
1	In one or two sentences, distinguish scripting from compiled programming. Give one automation example of each for your track (forensic: log triage vs a custom disk-imaging tool; cloud: bucket sync vs a custom storage engine).	1	1
2	Write the exact output of the snippet below.	1	1
3	Describe one realistic task that needs both a Python script and a Bash script in your track's pipeline. State what each script does and how they communicate (exit code, file, pipe).	2	1

```
files = ["a.log", "b.txt", "c.log"]
for f in files:
    if f.endswith(".log"):
        print("parse", f)
    else:
        print("skip", f)
```

Model answers: Q1: interpreted, quick to change, glue code, versus compiled for performance and large systems (0.5 for definition, 0.5 for valid example). Q2: parse a.log, skip b.txt, parse c.log (each on its own line). Q3: for example, Bash cron runs a Python parser and uses its exit code to decide whether to upload the report (0.5 for task, 0.5 for the interface between them).

Quiz 2 (Weeks 6-10), 3 marks, closed book, 15 minutes

Q	Question	CLO	Marks
1	**Forensic:** why is Node.js well suited to renaming thousands of evidence files? **Cloud:** why is Node.js well suited to a serverless function? Answer in two sentences.	2	1
2	List, in order, the steps a script takes to retrieve JSON from a REST API and store it safely (include at least five steps).	2	1
3	Identify the defect in the snippet below and give the one-line fix.	3	1

```
resp = requests.get(url, headers=headers)
data = resp.json()
print(data["data"]["score"])
```

Model answers: Q1: non-blocking I/O with async/await handles many file operations concurrently; small, event-driven handlers fit serverless. Q2: load credential from environment, send request with timeout, catch network exceptions, check status code, parse JSON defensively, write timestamped file, hash it, log it. Q3: no timeout and no status check, so an error body is parsed as data; fix: resp = requests.get(url, headers=headers, timeout=10); resp.raise_for_status() (also accept a .get() based fix for missing keys).

Quiz 3 (Weeks 11-15), 4 marks, closed book, 20 minutes

Q	Question	CLO	Marks
1	Give the correct order of instructions in a minimal Dockerfile for a Python script and explain in one sentence why dependency installation precedes the code copy.	2	1
2	Name one free, open-source tool that scans a container image for known vulnerabilities in a CI pipeline, and say what policy you would make it enforce.	2	1
3	**Forensic:** the function below opens an evidence file. **Cloud:** the function below calls a cloud API. Name the specific exception to catch and show the except line.	3	1
4	Why is a rotating file handler preferable to an unbounded log file in a long-running monitor? Give one further reason to keep a retention policy.	3	1

```
# Forensic version
def load(path):
    with open(path) as f:
        return f.read()

# Cloud version
def status(url):
    return requests.get(url, timeout=10).json()
```

Model answers: Q1: FROM, WORKDIR, COPY requirements, RUN pip install, COPY code, (USER), ENTRYPOINT; caching means code edits do not reinstall dependencies. Q2: Trivy (or Grype); fail the build on HIGH/CRITICAL fixable findings. Q3: except FileNotFoundError (or OSError), except requests.exceptions.RequestException. Q4: caps disk use and keeps files manageable; retention balances evidential or diagnostic value against storage and privacy.

Assignments

Assignment 1 (Weeks 1-5), 3 marks, CLO 1

Build a standard-library-only Python script that reads a supplied log (Forensic) or CSV (Cloud) line by line, extracts at least three fields, survives malformed lines, and prints or saves a summary.

1. Read the file line by line (csv module for CSV).
2. Extract at least three fields per line (Forensic: timestamp, source IP, event type; Cloud: resource ID, region, status).
3. Handle at least one malformed-line case and report how many were skipped.
4. Summary: counts per event type or per region and status; earliest and latest timestamp (Forensic).
5. Submit the .py file, a half-page explanation, and the GitHub link with at least three commits.

Rubric	Marks
Correct field extraction and summary output	1.5
Robust handling of at least one malformed input case	1
Clarity of the written explanation	0.5

> **Personalisation**
> Give each student a different input file by seeding a generator script with the roll number, or ask them to filter on a value derived from it. Two identical submissions then become obvious.

Assignment 2 (Weeks 6-10), 3 marks, CLO 2 (2) and CLO 3 (1)

Build a Node.js script that fetches JSON from a free public API (Forensic) or from a public API or local emulator (Cloud), with defensive error handling.

1. Use axios or built-in fetch with a timeout.
2. Parse and display at least two specific fields.
3. Wrap the network call in try/catch (or .catch()), logging a clear message on failure.
4. Save the data to a local file with a timestamped name.
5. Submit the script and a one-paragraph note on which failure scenario you tested and how it was handled, plus the GitHub link.

Rubric	Marks
Correct API call and field extraction	1.5
Demonstrated, working error handling for at least one failure scenario	1
Clarity and correctness of the note	0.5

Assignment 3 (Weeks 11-16), 4 marks, CLO 2 (2) and CLO 3 (2)

Capstone mini-project: containerize one of your earlier scripts, add structured logging and error handling, and build a GitHub Actions workflow that runs on every push.

1. Choose a base script (Assignment 1, Assignment 2 or the Week 11 workflow).

2. Replace all print/console output with Python logging or Winston at suitable levels; add at least one specific exception handler.
3. Author a correctly ordered, non-root Dockerfile and .dockerignore; confirm build and run locally.
4. Add a GitHub Actions workflow: checkout, install, smoke test, build the image; Week 15 adds the Trivy scan (recommended).
5. Submit the repository link and a README mapping each component to a CLO. The repository must show at least five commits and one reviewed pull request.

Rubric	Marks
Correct, working Dockerfile and successful local build/run	1.5
Appropriate structured logging and at least one handled exception	1.5
Working CI workflow that runs automatically on push (the commit history and pull-request evidence for CLO 3 are checked here)	1

One-Hour Class Task (Mid-Semester Practical), 10 marks

Item	Detail
Coverage	Weeks 1-8
Marks and CLOs	CLO 1 = 3, CLO 2 = 4, CLO 3 = 3 (as in both marks tables)
Format	Timed, individual, in the lab, no internet except documentation, no chat assistants

Each student receives a broken script (Forensic: log parser or hash checker; Cloud: CSV/resource processor) with no error handling, print statements only and **one logical bug**. Create a different bug per student group, for example off-by-one in a date range, wrong comparison operator, or a counter reset inside a loop.

1. Identify and fix the logical bug; write one sentence explaining the root cause.
2. Add at least one try/except around a genuinely fragile operation (file or network I/O).
3. Replace print() calls with at least one logging call at a suitable level.
4. Commit to Git with a meaningful message.
5. Submit the corrected script and a screenshot of a successful run showing the system clock.

Rubric (as approved)	Marks	CLO
Correct identification and repair of the logical bug	4	2
Appropriate, specific exception handling added	3	3
Correct use of scripting fundamentals throughout	3	1

Instructor note on CLO mapping

Finding a logical bug is an "Analyze" skill (CLO 3), and the approved table gives CLO 2 the largest share (4 marks). I kept the approved split so totals stay valid. To make the CLO 2 marks defensible, require the repair to include **redesign for reuse**: moving the fix into a function with parameters and no hard-coded paths or credentials. Please align the wording at the next outline revision. Primers for exceptions, logging, Bash exit codes and Git are now taught before the test (Weeks 2, 3, 5, 7).

Final Exam Blueprint (Weeks 1-16), 50 marks

Approved split: CLO 1 = 5, CLO 2 = 20, CLO 3 = 25. Suggested paper:

Section	Content	Marks	CLO
A. Q1	Short code-writing: parse or transform data (Python or Bash)	5	1
B. Q2	Design an API-driven fetch-evaluate-alert workflow with state and scheduling (diagram plus pseudo-code)	8	2
B. Q3	Write or correct a Dockerfile and explain ordering, volume mounts and secrets handling	6	2
B. Q4	Complete or debug a CI YAML (test, build, scan; pinning and permissions)	6	2
C. Q5	Debug a supplied script: read the traceback, locate and fix the logical and runtime errors	8	3
C. Q6	Design logging for a long-running tool: levels, rotation, retention, JSON fields, tamper-evidence or incident diagnosis	6	3
C. Q7	Write specific exception handling and a custom exception for a described failure	5	3
C. Q8	Git collaboration scenario: conflicting changes, pull request review, secret committed by mistake	6	3
	Total	**50**	5 / 20 / 25

Tailor Q2, Q5 and Q6 to the track: forensic cases (evidence integrity, chain of custody) for DFSP-5208 and operations cases (provisioning, capacity, outage diagnosis) for CCSP-5213.

Appendix A: One Repository Across Weeks 10-16

```
project/
  .github/workflows/ci.yml
  .gitignore            # .env, venv/, evidence/, *.log
  .dockerignore
  Dockerfile
  requirements.txt
  README.md             # stages and CLO mapping
  api_lookup.py         # Week 10
  workflow.py           # Weeks 11-13 (fetch, evaluate, alert, logging)
  tests/test_evaluate.py
  node/fetch.js         # Week 10 / Assignment 2
```

Each week adds to the same repository so that Assignment 3 is an assembly of work already done, not a new task.

Appendix B: Personalisation and AI-Use Policy for Evidence-Based Questions

The original scenario questions could be answered by pasting them into a chatbot. The Fall 2026 questions need artifacts from the student's own machine. Suggested rules to announce in Week 1:

- **Evidence rule:** every answer includes own terminal output, a screenshot with the system clock, a file hash, a log or a repository link.
- **Personal seed:** students derive a parameter from their roll number (threshold = last two digits, --top = last digit, rename prefix = roll number). A copied answer will not match the seed.
- **Spot viva:** the instructor asks 2-3 students per lab to explain one line live (2 minutes). Failing the viva voids the evidence.
- **Allowed AI use (suggested):** explaining an error message or a concept is allowed if the student can reproduce and explain the result; submitting generated code or output that the student cannot explain is academic misconduct under the university by-laws. The instructor should decide and state the exact rule.
- **Commit history:** a repository whose full solution arrives in one commit is flagged for a viva.

Appendix C: Pre-Term Checklist for the Instructor

1. Install and test Python 3.12+, Node LTS, Git, VS Code, Docker and WSL on a sample lab PC; confirm virtualization is enabled.
2. Check outbound access to PyPI, npm, GitHub, Docker Hub and your chosen API provider from the lab network.
3. Pick the cloud emulator (Moto recommended), install it and run the Week 10 script against it.
4. Create or verify a free-tier API key, or deploy the mock API for everyone.
5. Create a Discord or Slack test webhook for Week 11.
6. Pre-pull python:3.12-slim and share it by docker save if the lab bandwidth is limited.
7. Confirm the textbook chapter numbers for your edition and update the reading column.
8. Look up the current pinned version of aquasecurity/trivy-action and the current majors of the official GitHub actions.
9. Create a GitHub organization or Classroom, branch-protection template and repository template.
10. Prepare a broken-script bank (one logical bug each) for the class task, and sample logs or CSVs with malformed lines.
11. Decide and announce the AI-use rule and viva procedure.

Appendix D: Reference List

- Sweigart, A. *Automate the Boring Stuff with Python* (core text, DFSP-5208).
- Erl, T. *Cloud Computing: Concepts, Technology & Architecture* (core text, CCSP-5213).
- Severance, C. *Python for Everybody*.
- Garrels, M. *Bash Guide for Beginners* (free PDF).
- Casciaro, M. *Node.js Design Patterns*.
- Poulton, N. *Docker Deep Dive*.
- Sikorski, M. and Honig, A. *Practical Malware Analysis* (optional enrichment for forensic students, not a Week 10 reading).
- Lutz, M. *Learning Python*.
- Chacon, S. and Straub, B. *Pro Git* (free online).
- Official documentation: Python (logging, sqlite3, argparse), Node.js, requests, Docker, GitHub Actions, Trivy, Grafana Loki, Moto.